ANÁLISIS DEL RETO

Juan Pablo Martinez, 202611361, jp.martinezr1@uniandes.edu.co

Asignación de requerimientos: todo el reto lo desarrolló Juan Pablo Martínez. Se implementó el requerimiento individual 3 y los requerimientos grupales 4, 5 y 6. Los requerimientos individuales 1 y 2 no se implementaron porque le correspondían a otros integrantes y solo se desarrolló el requerimiento individual propio.

Carga de datos y tablas de hash

En la carga se lee el archivo CSV fila por fila. Cada fila se convierte en un registro con los tipos correctos (números como int o float y fechas como texto YYYY-MM-DD, que se puede comparar directamente). El registro se agrega a la lista principal de pedidos y, en la misma pasada, se ubica en varias tablas de hash pensadas para que cada requerimiento vaya directo a los pedidos que necesita sin recorrer la lista completa. Todas las tablas usan Linear Probing con factor de carga 0.5:

Tabla	Llave	Valor	Uso	Nº de llaves
pedidos_por_canal	Canal	Número de pedidos	Conteo por canal en la carga	3
pedidos_por_fecha	Fecha del pedido	Lista de pedidos de ese día	Requerimiento 3	730
promedio_producto_pais	Tupla (producto, país)	Lista de pedidos de la combinación	Requerimiento 4	60
pedidos_por_pais	País	Lista de pedidos del país	Requerimiento 5	5
pedidos_por_producto	Producto	Lista de pedidos del producto	Requerimiento 6	12

Escogimos Linear Probing porque en todas las tablas el número de llaves distintas es pequeño y se conoce de antemano: los pedidos repetidos no crean llaves nuevas, solo se agregan a la lista guardada como valor. Con un factor de carga de 0.5 las secuencias de sondeo son cortas, así que put, get y contains cuestan O(1) en promedio. Además, Linear Probing guarda todo en un solo arreglo, sin listas auxiliares por casilla como Separate Chaining, y su punto débil, la eliminación de llaves, no nos afecta porque nunca borramos. Por eso Separate Chaining no aportaba ninguna ventaja aquí.

Complejidad de la carga: por cada una de las n filas se hacen operaciones O(1) (add_last y las operaciones de las tablas), lo que da O(n). Al final se ordena la lista principal con Merge Sort por Amount para sacar los 5 primeros y los 5 últimos, lo que cuesta O(n log n). La complejidad total de la carga es O(n log n).

Sobre el ordenamiento: en todos los requerimientos usamos Merge Sort porque garantiza O(n log n) aun en el peor caso y es estable, así que los criterios de desempate

se respetan. Quick Sort puede llegar a $O(n^2)$ y, con los muchos valores repetidos de estos datos, nos dio problemas de recursión. Los algoritmos cuadráticos no eran viables para el archivo completo.

Requerimiento 3 (implementado por Juan Pablo [APELLIDOS])

Este requerimiento lista los pedidos de una fecha específica cuyo Amount está en un rango dado. Recibe la fecha y el Amount mínimo y máximo. Devuelve el tiempo de ejecución, el número de pedidos, las sumas de Amount, Marketing_Spend y aporte comercial, y los pedidos ordenados (si son más de 20, solo los 10 primeros y los 10 últimos).

Con la fecha se hace un get en pedidos_por_fecha, que es O(1), y se obtiene solo la lista de pedidos de ese día. Esa lista se recorre aplicando el filtro de Amount con los extremos incluidos y se acumulan las sumas; si k es el número de pedidos de la fecha, este paso es O(k). Luego se ordenan con Merge Sort los f pedidos que pasaron el filtro (Amount descendente, aporte descendente, Order_ID ascendente), en O(f log f). Sacar los 10 primeros y los 10 últimos es O(1).

Paso	**Complejidad**
Obtener los pedidos de la fecha (get en pedidos_por_fecha)	O(1)
Recorrer los k pedidos, filtrar por Amount y sumar	O(k)
Ordenar con Merge Sort los f pedidos filtrados	O(f log f)
Tomar los 10 primeros y los 10 últimos	O(1)
TOTAL ($k \approx n/730$)	O(k log k)

Requerimiento 4

Este requerimiento calcula, para una combinación de producto y país, el número de pedidos, los promedios de precio por caja, marketing, descuento y cajas, y muestra los 2 pedidos con mayor Amount.

Se arma la llave (producto, país) y se hace un get en promedio_producto_pais, que es O(1). Se recorren los m pedidos de esa combinación sumando los valores para los promedios, en O(m). Luego se ordenan con Merge Sort (Amount descendente, Marketing_Spend ascendente, Order_ID ascendente), en O(m log m), y se toman los 2 primeros.

Paso	**Complejidad**
Obtener los pedidos de la combinación (get con la llave (producto, país))	O(1)
Recorrer los m pedidos y sumar para los promedios	O(m)
Ordenar con Merge Sort	O(m log m)
Tomar los 2 primeros y calcular los promedios	O(1)

TOTAL (m ≈ n/60)	O(m log m)

Requerimiento 5

Este requerimiento encuentra los N productos con mayor recaudo para un país dentro de un rango de fechas. Muestra el total de pedidos, el número de productos distintos, el recaudo total y, para cada producto del top N, sus pedidos, cajas, recaudo, promedios de precio y descuento, y marketing.

Con un get en pedidos_por_pais (O(1)) se obtienen los p pedidos del país. Se recorren filtrando por fecha y cada pedido válido se agrupa en una tabla temporal (Linear Probing, llave producto) que acumula sus totales; cada operación es O(1), así que el recorrido es O(p). Después, value_set pasa los P productos a una lista, que se ordena con Merge Sort (recaudo descendente, cajas descendente, nombre ascendente) y se corta en los N primeros. Si N es mayor que el número de productos, se muestran todos.

Paso	**Complejidad**
Obtener los pedidos del país (get en pedidos_por_pais)	O(1)
Recorrer los p pedidos, filtrar por fecha y agrupar por producto	O(p)
Pasar a lista (value_set)	O(P)
Ordenar con Merge Sort	O(P log P)
Tomar el top N y calcular los promedios	O(N)
TOTAL (p ≈ n/5, P ≤ 12)	O(p)

Requerimiento 6

Este requerimiento encuentra los N países con mayor eficiencia de mercadeo (recaudo dividido entre marketing) para un producto en un rango de fechas. Muestra el total de pedidos, el número de países, el Amount y el Marketing_Spend totales y, por cada país del top N, sus pedidos, Amount, marketing, cajas, precio promedio y eficiencia.

Con un get en pedidos_por_producto (O(1)) se obtienen los q pedidos del producto. Se recorren filtrando por fecha y agrupando por país en una tabla temporal, en O(q). Luego se pasa a una lista con value_set y solo entran al ranking los países con marketing mayor que 0, a los que se les calcula la eficiencia. El ranking se ordena con Merge Sort (eficiencia descendente, Amount descendente, país ascendente) y se toman los N primeros.

Paso	**Complejidad**
Obtener los pedidos del producto (get en pedidos_por_producto)	O(1)
Recorrer los q pedidos, filtrar por fecha y agrupar por país	O(q)
Pasar a lista (value_set) y armar el ranking con la eficiencia	O(C)
Ordenar con Merge Sort	O(C log C)

Tomar el top N y calcular el precio promedio	O(N)
TOTAL (q ≈ n/12, C ≤ 5)	O(q)

Comparación experimental del Requerimiento 4 (Reto 1 vs. Reto 2)

Las dos versiones se corrieron en el mismo computador, con los mismos archivos (20, 40, 60, 80 y 100 % de los datos) y las mismas 5 consultas en cada tamaño: 70% Dark Bar / Germany, Truffle Gift Box / Australia, Hazelnut Milk Bar / India, Salted Caramel Bar / Brazil y Almond Crunch Bar / Japan. Se midió solo el tiempo del requerimiento, sin la carga ni la impresión. Los tiempos promedio fueron:

Tamaño de los datos	**Reto 1 (ms)**	**Reto 2 (ms)**	**Veces más rápido**
20 %	49.84	8.27	6.0
40 %	193.86	17.05	11.4
60 %	431.92	30.01	14.4
80 %	736.33	38.15	19.3
100 %	1183.58	49.05	24.1

El Reto 2 fue entre 6 y 24 veces más rápido, y la diferencia crece con el tamaño del archivo. En el Reto 1 hay que recorrer los n pedidos para encontrar los de la combinación. Además, los filtrados se guardaban en una lista enlazada que luego se recorría con get_element, que en esa estructura es O(i), y eso hace que el tiempo crezca más rápido que linealmente. En el Reto 2, el get por la tupla (producto, país) es O(1) y solo se recorren los m pedidos de la combinación, así que el tiempo depende de m y no de n. Por eso las consultas con pocos pedidos, como Hazelnut Milk Bar / India, tardan apenas unos milisegundos incluso con el archivo completo. El detalle de cada ejecución y la gráfica están en el archivo de Excel de la carpeta Docs.